

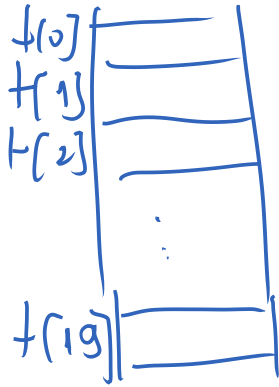
StructFds.

I. introduction

int t[20]; c'est comme int t[0], t[1], t[2], ..., t[19];

t[2] = 5;

printf("%d", t[6]);



II Definition

Une structure (appelée *enregistrement*) est un ensemble de variables qui ne sont pas forcément de même type.

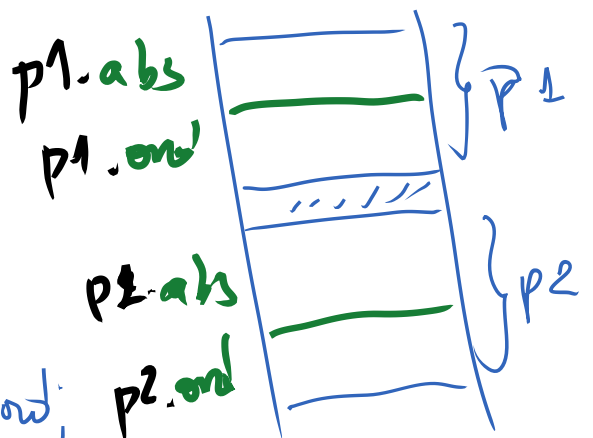
III Déclaration et Utilisation

```
1°) struct  
    { type1 champ1;  
      type2 champ2;  
      ...  
      typeN champN;  
    } nom_variables;
```

(nom variables séparés par des virgules)

Exemples:

```
a) struct  
    { int abs;  
      int ord;  
    } p1, p2;
```



c'est comme :

int p1.abs, p1.ord, p2.abs, p2.ord;

1) remplir p1.

printf("Entrez les infos de p1:");

scanf("%d %d", &p1.obs, &p1.ord);

2) Modifier l'ordonnée de p1 en mettant 9.

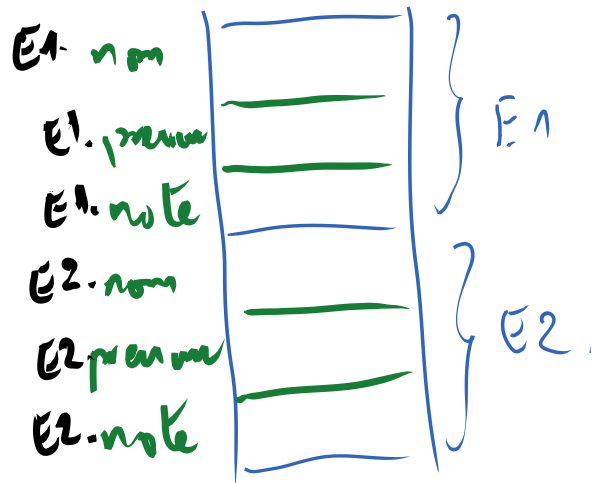
p1.ord = 9;

3) Afficher p1.

printf("Le contenu de p1 est a l'adresse: %d et ordonnee: %d\n",
p1.obs, p1.ord);

b) struct

```
{ char nom[20];  
  char prenom[15];  
  float note;  
  } E1, E2;
```



Saisir E1.

printf("Entrez les infos de E1:");

gets(E1.nom);

gets(E1.prenom);

scanf("%f", &E1.note);

fflush(stdin);

~~c) struct-
{ int coef;
 float note;
 } M1[10];~~

type def

~~struct { type1 champ1;~~

~~type N champ N;~~
~~{ param1, struct~~

~~{ type1 champ1;~~

~~type N champ N;~~

~~{ param1,~~

~~...)~~

}

typedef

struct

{ type1 champ1;

type2 champ2;

...

type N champ N;

} nouveau_type;

nouveau_type

nonvariable(s);

≠)

struct { type1 champ1;

type N champ N;

{ nonvariable(s);

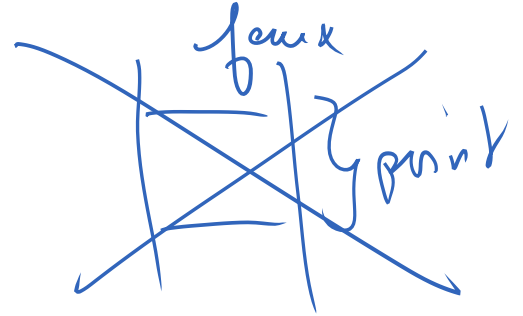
Exemples:

typedef int entier;

entier n; ≠) int n;

a/

a) typedef struct
 { int abs;
 int ord;
 } print;



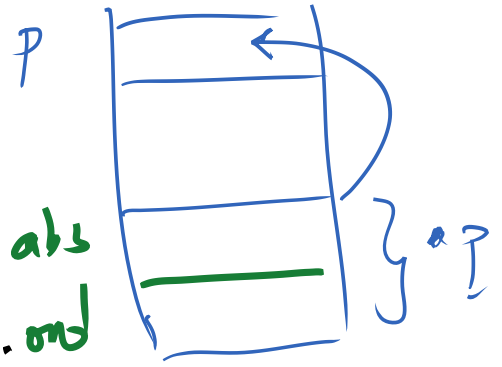
print p1, p2; $\Leftrightarrow$ struct
 { int ab;
 int ord;
 } p1, p2;

b) print *p;
 { p = (print*) malloc (sizeof(print));

(*p).abs equivalent
 $p \rightarrow \text{abs}$

(*p).ord equivalent
 $p \rightarrow \text{ord}$

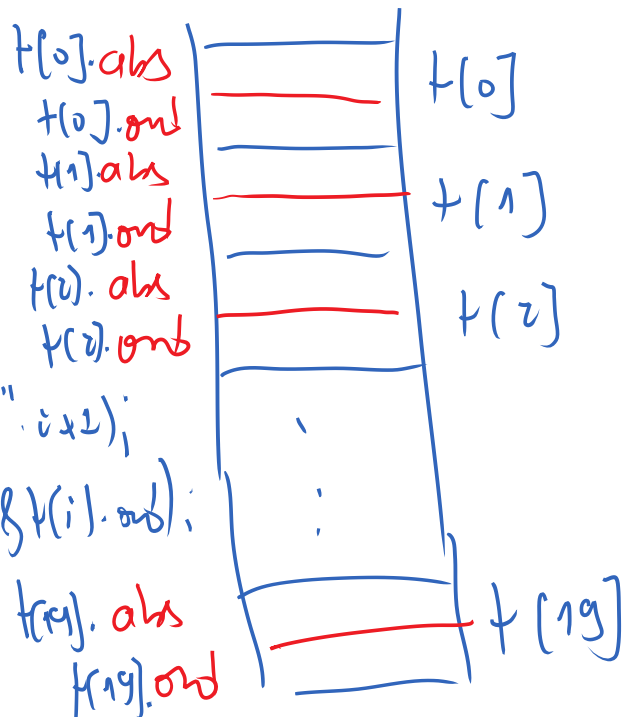
(*p).abs
 (*p).ord



b/ print t[20];

Remplir le tableau entier

```
for(i=0; i<20; i++)
{ printf("Entree d'elt numero %d: ", i+1);
  scanf("%d %d", &t[i].abs, &t[i].ord);
```



c) typedef struct
 { char nom[20];
 char prenom[15];

float note;
Étudiant;

- 1/ Saisir un tableau de N étudiants (N à saisir).
- 2/ Afficher parmi les étudiants ceux ayant plus que 12 et dont le nom commence par 'f' et 's'.

Requie:

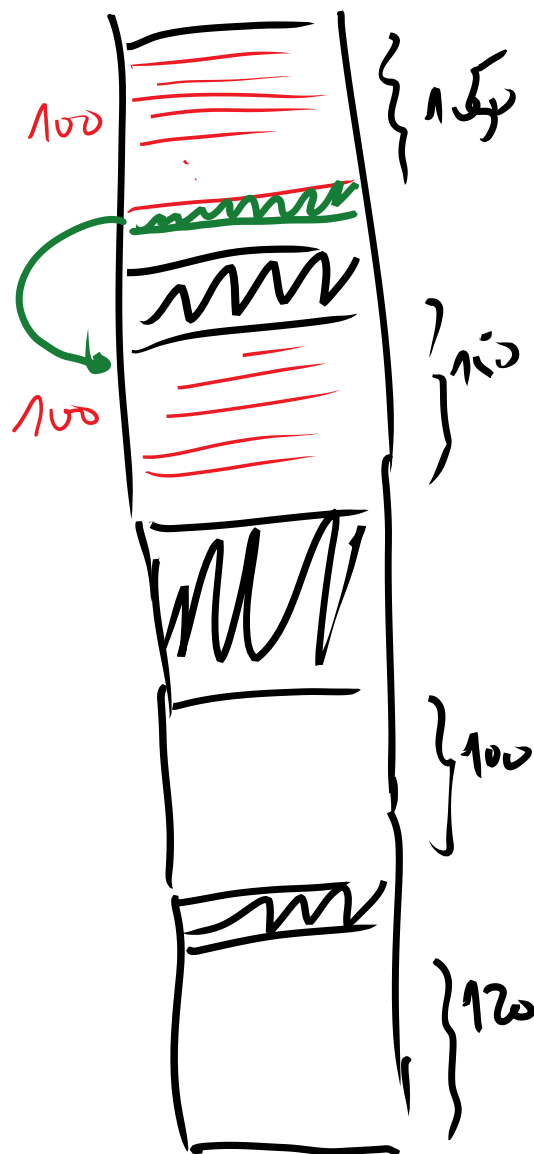
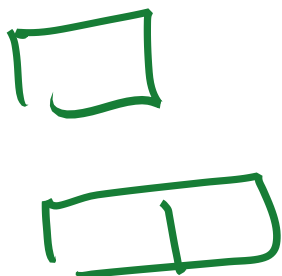
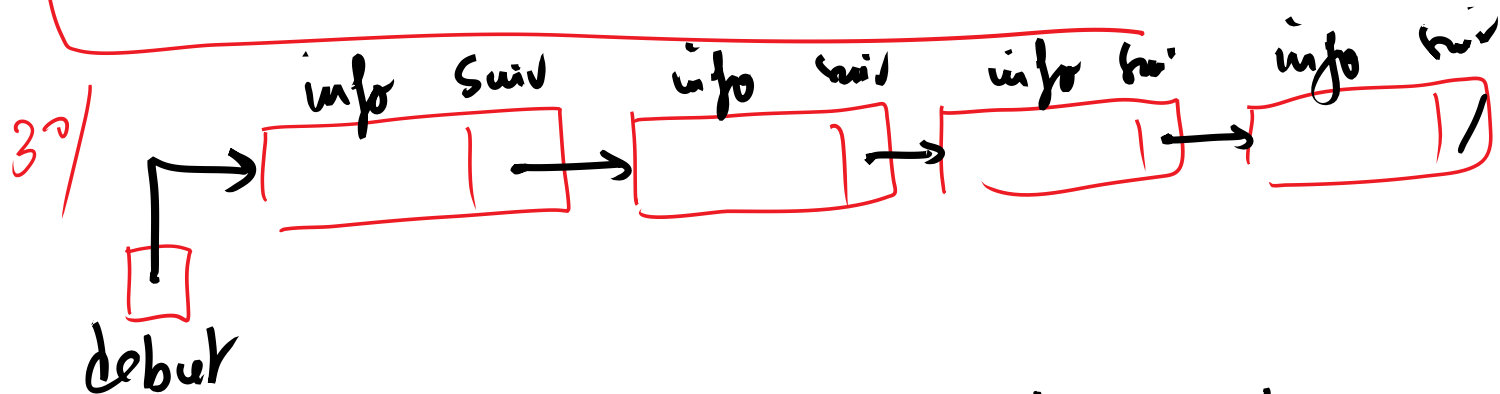
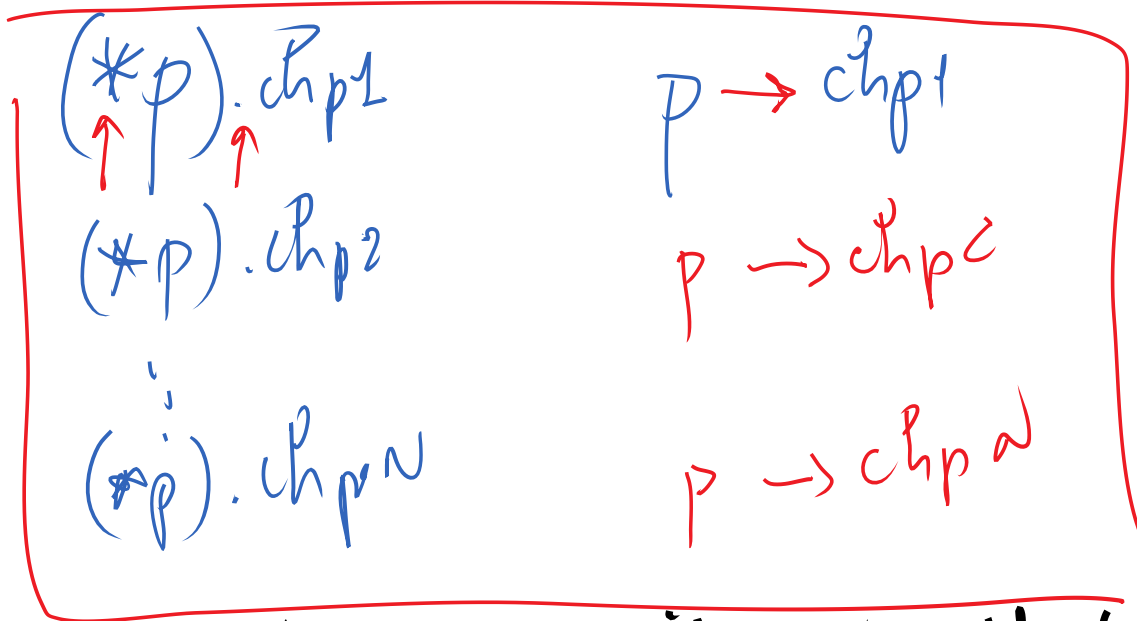
1/ ^{do} pour ("Entier N > 0:");
scanf("%d", &N);
^{while} (N <= 0);

E = (Étudiant*) malloc (N * sizeof(Étudiant));

```
for ( i = 0; i < N; i++ )  
{ printf( "Entier (Étudiant numero %d : ", i+1 );  
  gets( E[i].nom );  
  gets( E[i].prenom );  
  scanf( "%f", &E[i].note );  
}
```

2/ printf("Les étudiants ayant plus que 12 et dont le nom commence par f ou s sont:");

```
for ( i = 0; i < N; i++ )  
{ if ( ( E[i].note >= 12 ) && ( E[i].nom[0] == 'f' ||  
  E[i].nom[0] == 's' ) )  
  printf( "%s ", E[i].nom );  
}
```



typedef struct
 { type info;
~~void~~ suiv;
 } Noeud;

struct modele
{ type1 champ1;

⋮
typeN champN;

};

struct modele non_variable(s);

typedef struct modele non_variable_type;

typedef struct element
{ type info;
struct element *suiv;
};

Node X;

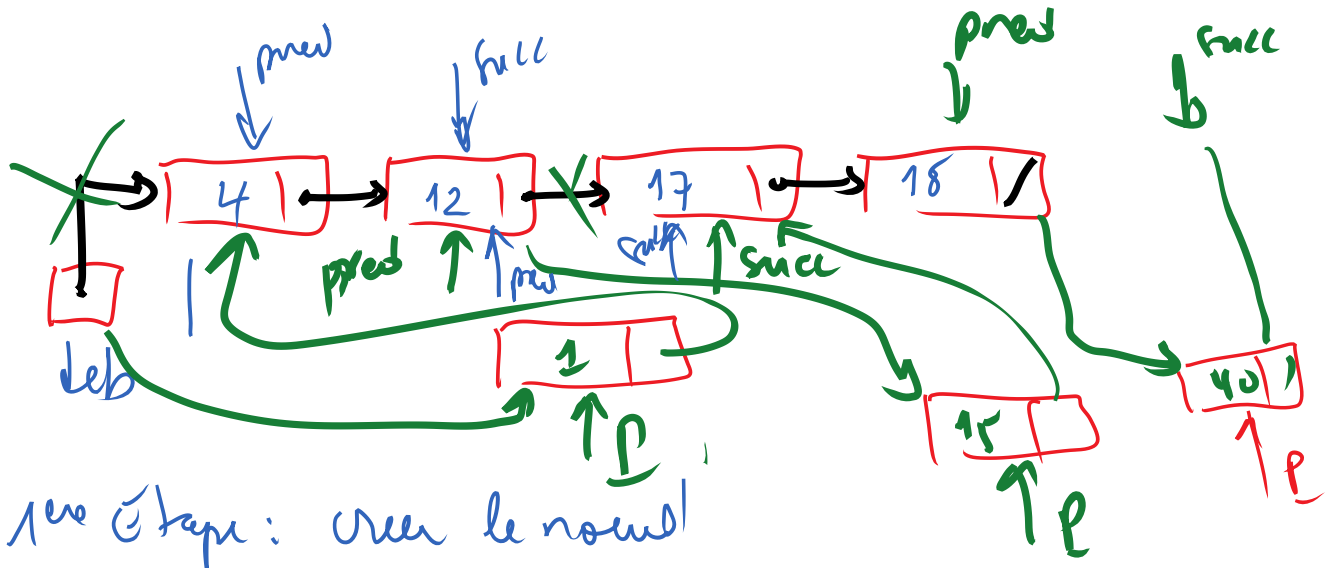
X.info
X.suiv

Node *p;

p = (Node*) malloc(sizeof(Node));

(*p).info
(*p).suiv

⇔ p → info
p → suiv



1^{re} étape: créer le nœud

2^{re} étape: Remplir le nœud

3^{re} étape: Insérer le nœud

1^{re} étape: $p = (\text{Nœud})\text{malloc}(\text{sizeof}(\text{Nœud}));$

2^{re} étape: $p \rightarrow \text{info} = x;$

3^{re} étape: 1^{er} Cas: au début

$p \rightarrow \text{suiv} = \text{deb};$
 $\text{deb} = p;$

2^{de} Cas: Au milieu

$\text{prev} \rightarrow \text{suiv} = p;$
 $p \rightarrow \text{suiv} = \text{succ};$

3^{de} Cas: À la fin

$\text{prev} \rightarrow \text{suiv} = p;$
 $p \rightarrow \text{suiv} = \text{null};$

typedef Nœud* ptr;

void insert (Node *deb, int x)

{ Node *p, *prev, *succ; // p, prev, succ;

p = (Node *) malloc(sizeof(Node));

p->info = x;

if (*deb == NULL)

{ p->next = NULL;
*deb = p;

else
-> if ((*deb->info > x)
{ p->next = deb;
*deb = p;

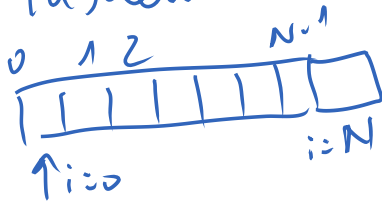
else
-> prev = deb;
succ = prev->next;
while ((succ != NULL) && (succ->info < x))
{ prev = succ;
succ = prev->next;

return

prev->next = p;
p->next = succ;

if (succ == NULL)
{ prev->next = p;
p->next = NULL;
else
{ prev->next = p;
p->next = succ;

tableau



1°) int i;

2°) i = 0

3°) i = N

4°) i++

5°) t[i]

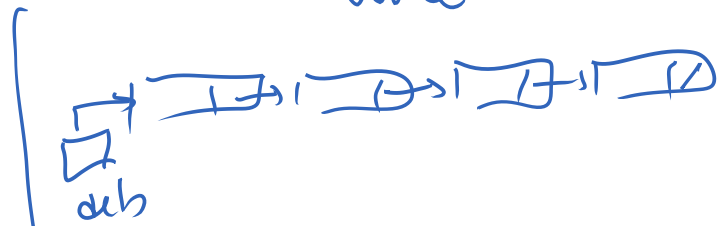
a) Affichage

```
i = 0;
while (i != N)
{ printf("%d", t[i]);
  i++;
}
```

b) parcours

```
i = 0;
while (i < N)
{ t[i] ???
  i++;
}
```

liste



1°) Node * p;

p = deb

p = NULL

p = p->next

p->info

p = deb;

while (p != NULL)

{ printf("%d", p->info);

p = p->next;

p = deb;

while (p != NULL)

{ p->info ???

p = p->next;

mt

sl